

FAA cooperativa

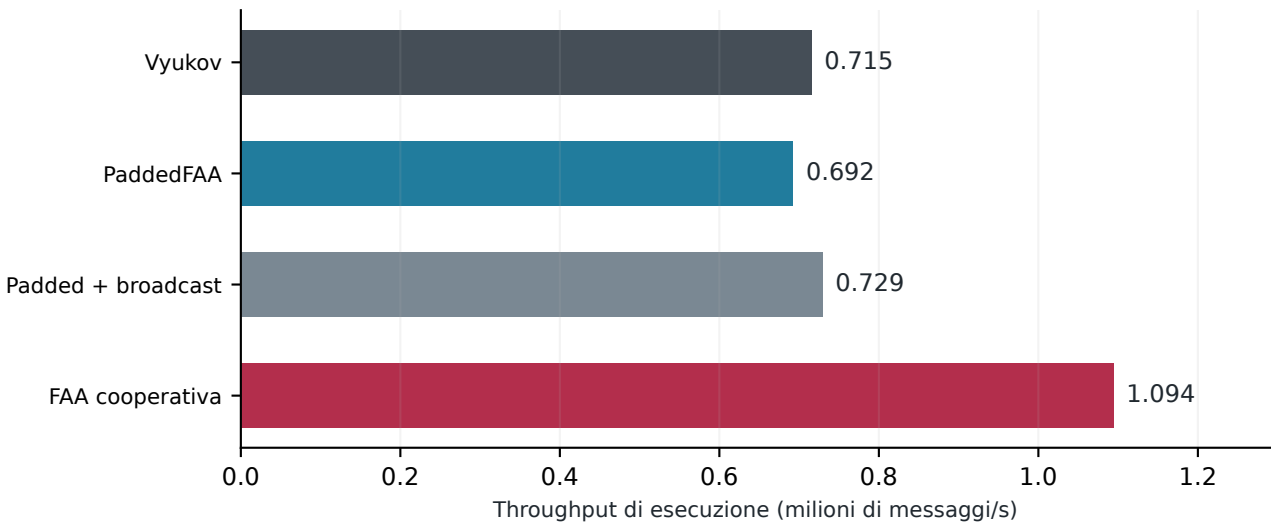
Confronto nella pipeline MoStream: risultati e valutazione tecnica

Conclusione principale

La nuova FAA migliora il throughput in un regime preciso: quattro worker, batch 8 e stage con poco o moderato calcolo. Non risulta una sostituzione universalmente migliore di Vyukov o della PaddedFAA normale.

Caso favorevole	Guadagno misurato
Throughput contro Vyukov	+53% rapporto 1.53 [1.49, 1.57]
Throughput contro PaddedFAA	+58% rapporto 1.58 [1.56, 1.60]
Tempo totale della pipeline	-8.8% contro Vyukov, incluso avvio e distruzione

Configurazione: 4 source, 4 transform, 4 sink; 4 worker su core fisici distinti; capacità 1024; batch 8; work=0; 40.000 messaggi totali.



Con batch 1 e quattro worker non emerge un vantaggio convincente. Con un solo worker la FAA perde in tutte le configurazioni misurate. Aumentando il calcolo per messaggio, il vantaggio con batch 8 scende a circa il 7%.

Base empirica: 336 misure temporizzate, sette ripetizioni per configurazione e variante, più 60 casi di verifica della pipeline. I risultati riguardano questa implementazione, macchina e carico sintetico.

Le quattro varianti

Il confronto riguarda sia la coda sia la sua integrazione nel runtime

Variante	Prenotazione cooperativa	Risveglio
Vyukov	CAS tramite try_push / try_pop	Wait queue originali
PaddedFAA	CAS nei try; slot padded	Wait queue originali
PaddedFAA-broadcast	Come PaddedFAA normale	Scansione degli actor bloccati
FAA cooperativa	FAA una volta, ticket mantenuto	Stessa scansione broadcast

Che cosa introduce la FAA cooperativa

Ogni actor conserva una PushOperation o PopOperation. Il primo tentativo prenota un ticket con fetch_add. Se lo slot non è pronto, l'actor si parcheggia mantenendo ticket e payload. Alla riattivazione riprova lo stesso ticket, senza spin interno e senza prenotarne un altro.

Il protocollo sequence conserva acquire/release sullo slot. La prenotazione non può essere abbandonata: un proprietario fermo può impedire il riuso della sua posizione. Questa variante non fornisce una garanzia lock-free e non supporta cancellazione o overflow dei contatori UInt64.

Perché cambia il risveglio

Con ticket persistenti, risvegliare un waiter arbitrario può lasciare parcheggiato il proprietario dello slot pronto. La prima integrazione risveglia quindi tutti gli actor bloccati nella direzione interessata. Registrazione dello stato BLOCKED, fence e ricontrollo evitano di perdere un evento durante il parcheggio.

PaddedFAA-broadcast usa la stessa politica di notifica della FAA, ma mantiene i CAS della coda originale. È il controllo necessario per non attribuire alla sola FAA un effetto dovuto ai risvegli.

Chiusura e durata di vita

Ogni producer segnala la fine dopo l'ultimo push completato. Le letture prenotate oltre l'ultimo messaggio diventano terminali solo dopo la chiusura definitiva. I comunicatori restano vivi fino al join di tutti i worker, comprese le notifiche finali. La ready queue resta MPMC CAS in tutte le varianti; il backend predefinito non viene sostituito.

Metodo sperimentale

Campagna locale del 21 settembre 2026; comandi e hash conservati nel manifest

Parametro	Valore
CPU	Intel Xeon Gold 5512U; 28 core / 56 thread; un socket
Compilazione	Mojo 1.0.0 (ed45d567); --Werror -O3
Pipeline	4 source -> 4 transform -> 4 sink; 10.000 messaggi/source
Worker e pinning	1 o 4 worker; CPU 0 oppure CPU 0,1,2,3, core fisici distinti
Capacità e batch	1024 slot; batch 1 oppure 8
Calcolo nel transform	work=0, 256, 2048 iterazioni dipendenti di mixing UInt64
Campionamento	12 configurazioni x 4 varianti x 7 ripetizioni = 336 misure

Due tempi distinti

Runtime: durata di Scheduler.start, inclusi dispatch/join, ready queue, calcolo, prenotazioni, parcheggio, ricontrolli e risvegli. Il throughput usa questo intervallo e conta i messaggi arrivati al sink.

Totale: intera chiamata run_cooperative, inclusa costruzione e distruzione di comunicatori e grandi code interne. La costruzione dell'oggetto Pipeline, Python e preparazione dei dati precedono entrambi i timer. Le wait queue originali sono allocate anche nelle varianti broadcast.

Protocollo e controlli

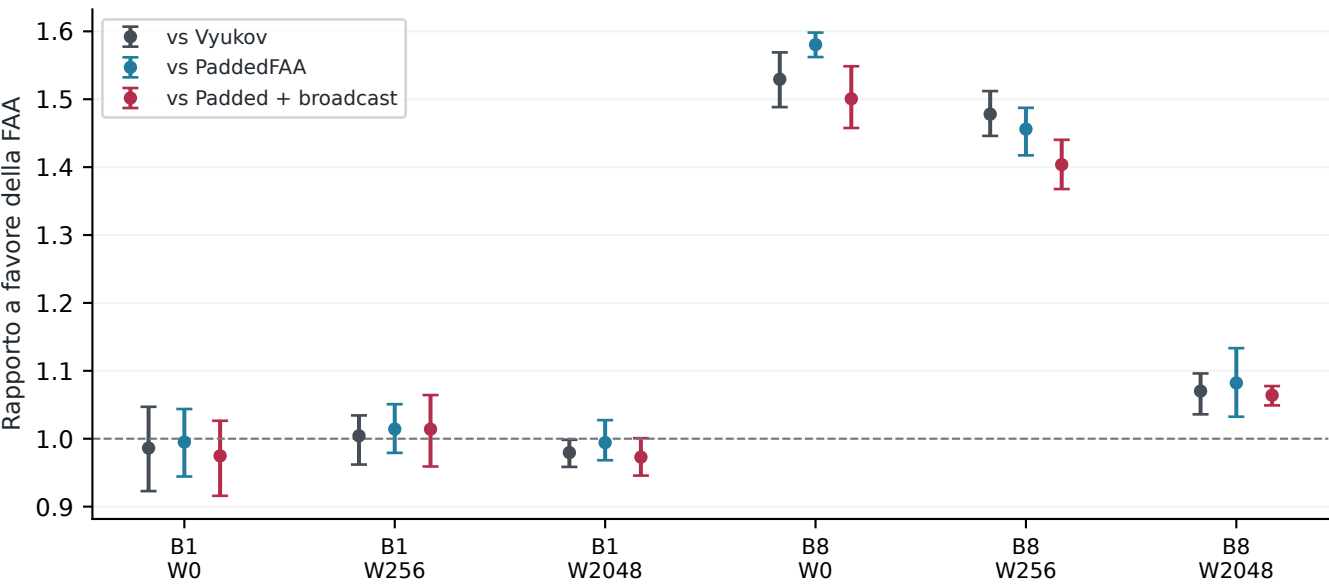
Un warm-up per caso/variante, escluso dalle statistiche. Sette blocchi con ordine randomizzato dei casi e delle varianti. Timeout e fallimenti interrompono la campagna; nessun outlier viene rimosso. Il checksum del calcolo viene verificato fuori dal tempo misurato.

Prima delle misure: 60 casi di verifica, inclusi input vuoto, due slot, più waiter che slot e transform che scarta tutti o parte dei messaggi. La verifica exact-once per ID è separata dalla temporizzazione. In ogni misura si controllano conteggio, checksum, campioni di latenza e assenza di ticket o payload pendenti alla fine.

Throughput con 4 worker

Capacità 1024; milioni di messaggi completati al secondo

Batch	Work	Vyukov	PaddedFAA	Padded + broadcast	FAA coop.
1	0	0.288	0.286	0.292	0.285
1	256	0.266	0.263	0.263	0.267
1	2048	0.207	0.204	0.208	0.203
8	0	0.715	0.692	0.729	1.094
8	256	0.581	0.590	0.611	0.858
8	2048	0.368	0.364	0.370	0.394



Nel grafico B indica il batch e W il work del transform, non il numero di worker (sempre quattro). Sopra 1 vince la FAA; le barre indicano IC95%.

Con batch 1 le differenze sono piccole: non c'è evidenza di una superiorità generale della FAA. Con batch 8, work=0 e work=256, il vantaggio resta anche rispetto al controllo con identici risvegli. Quando work=2048, il guadagno si riduce sensibilmente.

Gli intervalli sono bootstrap percentile su rapporti temporali accoppiati in log, con 4.000 ricampionamenti. Sono intervalli individuali, senza correzione per confronti multipli; sette ripetizioni rendono questa una campagna esplorativa, non una prova universale.

Batch e parallelismo

Un confronto favorevole fra code non implica scalabilità lineare del runtime

Worker	Batch	Work	Vyukov Mmsg/s	FAA Mmsg/s	FAA / Vyukov
1	1	0	0.667	0.624	0.93
1	1	2048	0.152	0.150	0.99
1	8	0	0.967	0.854	0.88
1	8	2048	0.160	0.158	0.99
4	1	0	0.288	0.285	0.99
4	1	2048	0.207	0.203	0.98
4	8	0	0.715	1.094	1.53
4	8	2048	0.368	0.394	1.07

Un solo worker

Con un solo worker non esiste competizione simultanea fra worker sui contatori della coda. La FAA aggiunge gestione dello stato persistente senza eliminare CAS falliti fra worker. Nei sei casi misurati risulta più lenta di Vyukov e PaddedFAA, con penalità più visibile nello stage leggero.

Il batch conta più del semplice cambio di coda

Con quattro worker e work=0, portare il batch da 1 a 8 aumenta il throughput anche per Vyukov (da 0.288 a 0.715 Mmsg/s). Con FAA si passa da 0.285 a 1.094 Mmsg/s. Il batch ammortizza attivazioni e notifiche; non raggruppa più prenotazioni in una sola FAA.

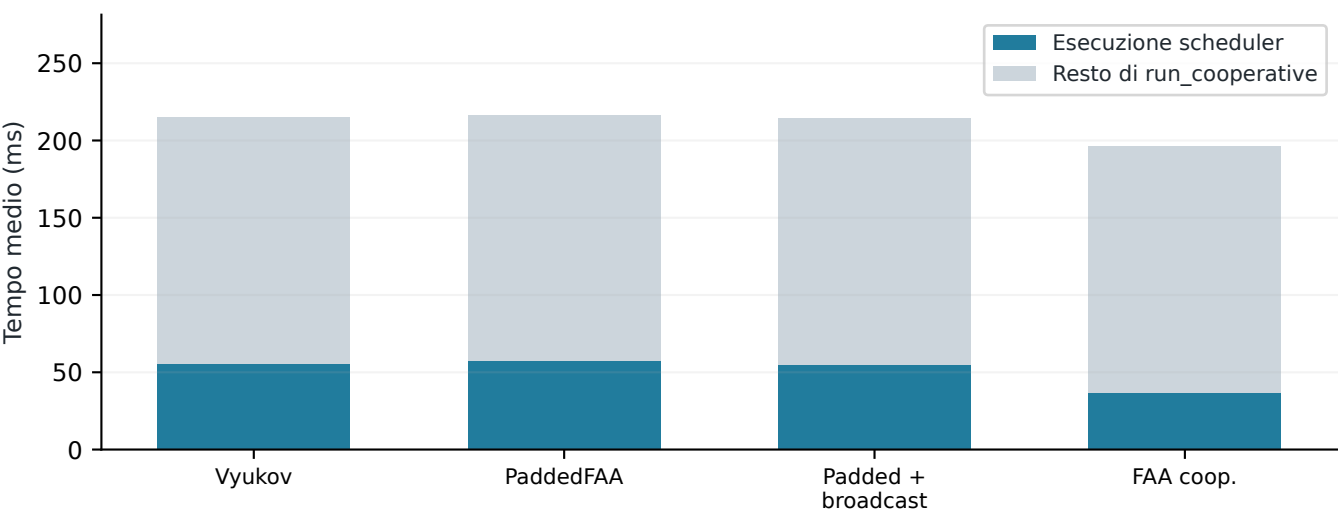
Più worker non sono sempre meglio

Con batch 1 e stage leggero, un worker supera quattro worker in entrambe le code. Aumentare il parallelismo introduce costi condivisi del runtime. La ready queue, gli stati degli actor e le notifiche sono candidati da profilare: i tempi da soli non consentono di attribuire il costo a una singola struttura o istruzione.

Quando il calcolo dello stage cresce, una frazione maggiore del lavoro non dipende dalla coda: il margine osservato fra le code si restringe. Per scegliere la configurazione conviene confrontare insieme numero di worker, batch, throughput e latenza.

Tempo totale e latenza

Quattro worker, batch 8: distinguere esecuzione e costo della chiamata completa



Work	Totale Vyukov ms	Totale FAA ms	p99 Vyukov us	p99 FAA us
0	215.23	196.22	1506	973
256	228.24	205.93	1863	1291
2048	269.06	260.22	3162	2986

Con work=0, la sola esecuzione scende da 55.96 a 36.59 ms. La chiamata completa passa da 215.23 a 196.22 ms: riduzione del 8.8%. Il +53% di throughput non significa +53% di prestazione complessiva per una pipeline breve.

Come leggere il p99

La latenza parte dalla creazione del messaggio nella source e termina nel sink, includendo accodamento, scheduling e calcolo. Si campiona un ID ogni 256: 157 campioni per esecuzione da 40.000 messaggi. Il valore riportato è la media dei p99 delle sette esecuzioni, non il p99 dell'intero flusso.

Con così pochi campioni per esecuzione, la coda estrema è descritta da pochissime osservazioni. Questi p99 sono indicativi; non dimostrano un vincolo di latenza o un obiettivo di servizio. Il batch va valutato anche per la latenza, non soltanto per il throughput.

Valutazione tecnica

Relazione sulle condizioni in cui la nuova variante offre un vantaggio

Risultato sostenuto dalle misure

La FAA cooperativa è una candidata preferibile quando la pipeline usa più worker, il batch ammortizza il costo dello scheduler e il lavoro per messaggio è limitato. Nella configurazione a quattro worker e batch 8 il vantaggio sul throughput è ripetuto in tutti i tre carichi, ma passa da circa il 53% al 7% rispetto a Vyukov.

Interpretazione del vantaggio

Il mantenimento del ticket elimina i tentativi CAS ripetuti per assegnare una posizione e distribuisce l'attesa sugli slot prenotati. Il controllo PaddedFAA-broadcast mantiene il medesimo protocollo di risveglio: con $\text{work}=0$ e batch 8 la FAA conserva un rapporto di circa 1.50 rispetto a quel controllo. Il vantaggio non è quindi spiegato soltanto dal passaggio dalle wait queue al broadcast.

Resta un'interpretazione algoritmica, non una misura diretta del numero di CAS falliti o dei trasferimenti di cache line. Non sono stati raccolti contatori hardware, consumo energetico o un profilo dettagliato. FAA conserva contesa sui contatori globali e dipendenze dai proprietari dei ticket; il broadcast aggiunge scansioni e possibili risvegli superflui.

Quando non sceglierla

Non emerge un motivo prestazionale per preferirla con un solo worker o con batch 1 in questa pipeline. Non va scelta come sostituzione trasparente se il sistema richiede cancellazione di operazioni prenotate o garanzie lock-free. Il backend predefinito resta Vyukov.

Portata della conclusione

La campagna riguarda un socket, una capacità, una topologia e un transform sintetico. Non stabilisce una classifica contro Rigtorp, SCQ, NBLFQ, wCQ o tutte le altre code presenti nel repository: qui non sono state misurate nello stesso runtime e con lo stesso protocollo.

Il circa 2x osservato nel precedente scheduler round-robin non si trasferisce automaticamente al runtime reale. I due esperimenti hanno topologia e costi diversi. La raccomandazione è sperimentare la FAA con batch configurabile sul carico applicativo effettivo, mantenendo la baseline e verificando throughput, latenza e durata totale.

Dati e riproducibilità

Rapporti di tempo accoppiati: sopra 1 vince FAA-cooperative

Worker	Batch	Work	vs Vyukov [IC95%]	vs PaddedFAA [IC95%]
1	1	0	0.93 [0.92, 0.95]	0.94 [0.93, 0.95]
1	1	256	0.97 [0.96, 0.98]	0.96 [0.95, 0.97]
1	1	2048	0.99 [0.98, 0.99]	0.99 [0.98, 1.00]
1	8	0	0.88 [0.87, 0.89]	0.87 [0.86, 0.89]
1	8	256	0.94 [0.94, 0.95]	0.93 [0.93, 0.94]
1	8	2048	0.99 [0.98, 0.99]	0.98 [0.98, 0.99]
4	1	0	0.99 [0.92, 1.05]	1.00 [0.94, 1.04]
4	1	256	1.00 [0.96, 1.03]	1.01 [0.98, 1.05]
4	1	2048	0.98 [0.96, 1.00]	0.99 [0.97, 1.03]
4	8	0	1.53 [1.49, 1.57]	1.58 [1.56, 1.60]
4	8	256	1.48 [1.45, 1.51]	1.46 [1.42, 1.49]
4	8	2048	1.07 [1.04, 1.10]	1.08 [1.03, 1.13]

Fonti interne

Benchmarks/RealCooperativeQueueBenchmark/results/ contiene results.csv (misure grezze), summary.csv (aggregati), manifest.json (ambiente, comandi, parametri e hash) e RESULTS.md (tabella completa). Il carico e le verifiche sono in Tests/test_cooperative_pipeline.mojo.

Questo PDF verifica l'hash del CSV e dei sorgenti, la completezza della matrice e ricalcola medie e intervalli dai dati grezzi prima di generare le pagine. Non unisce i dati del vecchio microbenchmark.

Comandi dalla radice del repository

```
python3 Benchmarks/RealCooperativeQueueBenchmark/run.py
python3 Documentazione/CooperativeFAA/genera_pdf.py
```

Per nuove campagne usare --output con una directory distinta. Sono richiesti Mojo 1.0, Python e matplotlib. Per riprodurre esattamente questa relazione occorrono i sorgenti identificati dal manifest.

SHA256 di results.csv:
f59d3ffacb7c03a676b6f777f7133813
f76744e934e41459986334dde12ac7f3